ALGORITHMIQUE EN BASH · JOUR 3 SUR 5 · MERCREDI 21 OCTOBRE

Mercredi · TP : le jeu du pendu

Toute la journée sur un seul programme : le jeu du pendu. L'ordinateur cache un mot, vous proposez des lettres une par une. Chaque lettre absente du mot compte comme une erreur. À 7 erreurs, c'est perdu.

On le construit en 5 étapes. À chaque étape, le script tourne et affiche quelque chose qu'on peut vérifier. **Ne passez pas à l'étape suivante tant que la vôtre n'affiche pas la même chose que l'exemple.** Et gardez une copie de chaque étape qui marche (`cp pendu.sh pendu_etape2.sh`), pour pouvoir revenir en arrière.

Étape 1 · Parcourir le mot lettre par lettre

Créez `pendu.sh`. Mettez le mot dans une variable : `mot="boucle"`. Affichez sa longueur, puis chaque lettre avec son indice.

Vous avez tout ce qu'il faut dans `mardi_voyelles.sh` : `${#mot}` donne la longueur, `${mot:i:1}` donne la lettre à l'indice `i`, et la boucle `for (( i = 0; i < ${#mot}; i++ ))` passe sur tous les indices.

Exemple d'exécution

```
$ ./pendu.sh
Le mot fait 6 lettres
Lettre 0 : b
Lettre 1 : o
Lettre 2 : u
Lettre 3 : c
Lettre 4 : l
Lettre 5 : e
```

Étape 2 · Cacher les lettres pas encore trouvées

Ajoutez une variable `trouvees="bo"` : ce sont les lettres que le joueur a déjà trouvées (pour l'instant, on l'écrit à la main). Le script doit maintenant afficher le mot avec un `_` à la place des lettres qui ne sont pas dans `trouvees`.

L'idée : on part d'une chaîne vide `affichage=""`, et dans la boucle on ajoute à la fin, pour chaque lettre du mot, soit la lettre, soit un `_`. Pour savoir si une lettre est dans `trouvees` :

```
if [[ $trouvees == *"$lettre"* ]]; then
```

Les `*` veulent dire « n'importe quoi avant, n'importe quoi après ». Le test est vrai si `$lettre` apparaît quelque part dans `$trouvees`.

Exemple d'exécution

```
$ ./pendu.sh
Mot : b o _ _ _ _
```

Changez `trouvees` en `"bue"`, puis en `""`, et vérifiez que l'affichage suit.

Étape 3 · La boucle de jeu

Maintenant, c'est le joueur qui donne les lettres. Remettez `trouvees=""`, et enveloppez tout votre code de l'étape 2 dans une boucle qui tourne jusqu'à la victoire :

```
while true; do
    # 1. fabriquer et afficher le mot caché (votre code de l'étape 2)
    # 2. si l'affichage ne contient plus de _ : afficher « Gagné ! » et break
    # 3. demander une lettre avec read, et l'ajouter à la fin de trouvees
done
```

Deux aides pour cette étape :

- `[[ $affichage != *_* ]]` est vrai quand il n'y a plus aucun `_` dans l'affichage ;
- `break` sort de la boucle `while`.

Attention : `affichage=""` doit être **dans** la boucle `while`. Sinon, le mot caché s'allonge à chaque tour.

Exemple de partie

```
$ ./pendu.sh
Mot : _ _ _ _ _ _
Une lettre : b
Mot : b _ _ _ _ _
Une lettre : o
Mot : b o _ _ _ _
Une lettre : u
Mot : b o u _ _ _
Une lettre : c
Mot : b o u c _ _
Une lettre : l
Mot : b o u c l _
Une lettre : e
Mot : b o u c l e
Gagné !
```

Étape 4 · Compter les erreurs

Ajoutez une variable `erreurs=0`. Après chaque lettre proposée :

- si la lettre est dans le mot, affichez « Bien vu ! » ;
- sinon, affichez « Raté ! » et ajoutez 1 à `erreurs`.

Le test est le même qu'à l'étape 2, sauf qu'on cherche la lettre proposée dans `$mot`.

Remplacez ensuite `while true` par `while (( erreurs < 7 ))` : la partie s'arrête toute seule à 7 erreurs. Après la boucle, affichez « Perdu ! » et le mot. Un piège : si le joueur gagne, il ne faut pas afficher « Perdu ! ». Remplacez le `break` de la victoire par `exit`, qui arrête tout le script.

Une partie gagnée

```
$ ./pendu.sh
Mot : _ _ _ _ _ _   (erreurs : 0/7)
Une lettre : e
Bien vu !
Mot : _ _ _ _ _ e   (erreurs : 0/7)
Une lettre : z
Raté !
Mot : _ _ _ _ _ e   (erreurs : 1/7)
Une lettre : b
Bien vu !
Mot : b _ _ _ _ e   (erreurs : 1/7)
Une lettre : o
Bien vu !
Mot : b o _ _ _ e   (erreurs : 1/7)
Une lettre : u
Bien vu !
Mot : b o u _ _ e   (erreurs : 1/7)
Une lettre : c
Bien vu !
Mot : b o u c _ e   (erreurs : 1/7)
Une lettre : l
Bien vu !
Mot : b o u c l e   (erreurs : 1/7)
Gagné !
```

Fin d'une partie perdue

```
$ ./pendu.sh
[...]
Raté !
Mot : _ _ _ _ _ _   (erreurs : 6/7)
Une lettre : j
Raté !
Perdu ! Le mot était : boucle
```

À ce stade, vous avez un vrai jeu. Faites-le tester à votre voisin.

Étape 5 · Un mot tiré au hasard

Remplacez `mot="boucle"` par un tableau d'au moins 6 mots, sans accents. Tirez-en un au hasard :

```
mots=(algorithme boucle variable fonction tableau terminal)
indice=$(( RANDOM % ${#mots[@]} ))
mot=${mots[indice]}
```

`RANDOM` donne un nombre au hasard. Le `%` (le reste de la division) le ramène entre 0 et le nombre de mots moins 1, donc toujours un indice valide.

Bonus, pour ceux qui ont fini

Dans l'ordre que vous voulez :

- **Les saisies bizarres.** Tapez `AB`, `3` ou rien du tout : aujourd'hui, ça compte comme une erreur. Refusez-les sans compter d'erreur. `[[ $choix =~ ^[a-z]$ ]]` est vrai seulement pour une seule lettre minuscule, et `continue` repart au début de la boucle.
- **Les lettres déjà jouées.** Si le joueur propose deux fois `z`, il prend deux erreurs. Prévenez-le à la place (« déjà jouée »), sans compter d'erreur.
- **Le dessin.** Dessinez le pendu en ASCII, une partie de plus à chaque erreur.

- **Une fonction.** Rangez le code qui fabrique `affichage` dans une fonction `masque`, et appelez-la dans la boucle.

Ce qui sera regardé en fin de journée

Votre jeu doit aller jusqu'au bout d'une partie gagnée et d'une partie perdue sans planter. C'est tout ce qui est demandé. Les bonus viennent ensuite, si vous avez le temps.